MAPÚA MALAYAN DIGITAL COLLEGE

A college under Mapúa Malayan Colleges Laguna

This template is an exclusive property of **Mapua-Malayan Digital College** and is protected under **Republic Act No. 8293**, also known as the *Intellectual Property Code of the Philippines* (IP Code). It is provided solely for educational purposes within this course. Students may use this template to complete their tasks but may not **modify**, **distribute**, **sell**, **upload**, or **claim ownership** of the template itself. Such actions constitute copyright infringement under **Sections 172, 177,** and **216** of the IP Code and may result in legal consequences. Unauthorized use beyond this course may result in legal or academic consequences.

Additionally, students must comply with the **Mapua-Malayan Digital College Student Handbook**, particularly with the following provisions:

*** Offenses Related to MMDC IT:**

- Section 6.2 – Unauthorized copying of files
- Section 6.8 – Extraction of protected, copyrighted, and/or confidential information by electronic means using MMDC IT infrastructure

*** Offenses Related to MMDC Admin, IT, and Operations:**

- Section 4.5 – Unauthorized collection or extraction of money, checks, or other instruments of monetary equivalent in connection with matters pertaining to MMDC

Violations of these policies may result in disciplinary actions ranging from **suspension** to **dismissal**, in accordance with the Student Handbook.

For permissions or inquiries, please contact MMDC-ISD at isd@mmdc.mcl.edu.ph.

MAPÚA MALAYAN DIGITAL COLLEGE

A college under Mapúa Malayan Colleges Laguna

MO-IT110 Object-Oriented Programming

Team Information

Team Details	
Team Name	
Team Members and Roles (if applicable)	Joemel Bataga
	Maria Alissa Bernales
	Grazielle Felice Canoza
	Shane Rivera
Worksheet Instructions Link	MO-IT110 | OOP | Milestone 1 Worksheet Instructions

MAPÚA MALAYAN DIGITAL COLLEGE

A college under Mapúa Malayan Colleges Laguna

MO-IT110 Object-Oriented Programming

CRC Cards

Select a Core Principle to view associated CRC Cards (Click the cell below):

Encapsulation	Abstraction	Inheritance	Polymorphism

ENCAPSULATION

Model Layer

Class Name	Responsibilities	Collaborators	Design Intent
Person	- Store basic personal identity information (first name and last name) - Provide controlled access to compensation information used in payroll computation	*Employee (inherits from Person)	Personal identity data is grouped into a dedicated class, allowing reuse and preventing duplication of name-related fields across the system.
Employee	- Store employee profile information - Maintain links to related employee data such as contact, government IDs, position, and compensation - Provide controlled access to employee-related information	Person ContactInfo GovernmentID Position Compensation Attendance (linked through attendance records) Payroll (payroll computed for employee)	Employee acts as the central model that encapsulates related information through composition, keeping the employee profile organised and reducing data redundancy.
ContactInfo	- Store employee contact details such as address and phone number - Provide access to contact information	Employee	Contact details are separated from the main employee profile to keep related data grouped and easier to maintain.
GovernmentID	- Store statutory identification numbers (SSS, PhilHealth, Pag-IBIG, TIN) - Provide access to government identification data	Employee	Government identifiers are encapsulated in a single class, improving clarity and making updates to statutory information easier to manage.

Position	- Store job-related information such as job title and supervisor - Provide access to position details	Employee	Job-related data is isolated from personal and compensation data, supporting clear separation of employee concerns.
Compensation	- Store pay-related values such as salary, hourly rate, and allowances - Provide access to compensation information used in payroll computation	Employee Payroll	Compensation details are grouped into their own class to ensure all pay-related data is managed consistently.
Attendance	- Store individual attendance records including date, time-in, and time-out - Associate attendance records with an employee	Employee Payroll	Attendance records are encapsulated separately to prevent payroll logic from directly managing raw time data.
Payroll	- Store payroll results for a specific period - Store computed values such as gross pay, deductions, and net pay	Employee Compensation	Payroll serves as a structured container for payroll results. Employee and compensation data are accessed in a read-only manner, ensuring payroll output does not modify core employee or compensation records.
Account	- Store user login credentials and role information - Provide controlled access to authentication-related data	Admin AuthService	Access to sensitive account data is controlled through read-only methods, supporting secure handling of credentials.
Admin	- Represent an administrative user in the system - Store admin-specific identification details	Employee Account	Admin extends the Employee model to reuse encapsulated employee identity and profile data while adding role-specific identification details.

Encapsulation in the Model layer is achieved by grouping related data into dedicated classes such as ContactInfo, GovernmentID, Compensation, Attendance, and Payroll. Each class is responsible for managing its own data and provides controlled access through methods, preventing direct manipulation of internal fields. This design improves clarity, reduces redundancy, and supports maintainability of the payroll system.

Service & Logic Layer			
Class Name	**Responsibilities**	**Collaborators**	**Design Intent**
DataLoaderService	- Load all required system data for payroll processing (e.g., employees and attendance) - Coordinate reading multiple data files and producing usable collections	CSVReader Employee Attendance	This class encapsulates the data-loading workflow so the rest of the system can simply call “load data” without needing to know file paths, parsing steps, or CSV details.
AttendanceService	- Compute total hours worked for an employee over a given time period (monthly / weekly) - Process attendance records into values usable by payroll computation - Validate attendance date ranges and missing records using flexible range-based checks (e.g., per-date or per-period validation)	Attendance Employee PayrollService	Attendance calculations are encapsulated here so payroll logic doesn’t directly manipulate raw time-in/time-out data. This keeps time computation consistent and easier to maintain if rules change.

PayrollService	- Compute gross pay using employee compensation data and hours worked - Compute deductions and net pay using the system's deduction rules - Produce payroll results in a consistent format (e.g., Payroll object / numeric outputs)	Employee Compensation AttendanceService Payroll DeductionsUtil	Payroll business rules are centralized in one service so calculations are consistent across the system. Other layers (controller/view) only request results instead of implementing formulas themselves. This supports maintainability and reduces duplication, one of the main reasons a service layer is used.
AuthService	- Authenticate users based on provided login credentials - Determine user role/type (e.g., admin vs employee) for correct access flow - Provide a controlled way for controllers to validate login without exposing credential-checking details	Account UserRole CSVReader / DataLoaderService (if accounts are loaded from CSV)	AuthService encapsulates authentication rules and credential checking so controllers do not directly handle passwords or login validation logic. This keeps security-related logic centralized and easier to manage consistently.
EmployeeService	- Retrieve and manage employee-related information (e.g., search employee, view profile details) - Provide employee data to other parts of the system in a structured way - Coordinate employee list access so controllers don't directly manipulate raw collections	Employee ContactInfo GovernmentID Position Compensation DataLoaderService	EmployeeService encapsulates employee-related operations and access to employee records, reducing duplication of search/filter logic across controllers and keeping employee handling consistent.
ReportService	- Generate payroll-related reports (payslips, summaries, exports) using computed payroll data - Coordinate report formatting and file output requests (e.g., export to CSV) - Provide a single place for report generation so controllers only request "generate report" rather than building outputs themselves	Payroll Employee PayrollService (source of computed payroll results) CSVWriter (for export/output)	ReportService encapsulates report generation workflows (what data goes into a report and how it is output), keeping presentation/output concerns separated from payroll computation and controller flow.

Encapsulation in the Service & Logic layer is achieved by isolating business rules and computations into dedicated classes such as AttendanceService and PayrollService. These classes expose simple public methods for calculating hours and payroll results while hiding internal calculation details (e.g., formula steps and deduction computations). This prevents controllers and views from directly implementing payroll logic, improves maintainability, and ensures consistent computation across the system.

Controller Layer			
Class Name	**Responsibilities**	**Collaborators**	**Design Intent**
AdminController	- Handle admin-specific workflows (admin menu actions) - Coordinate calls to services for admin tasks (e.g., loading data, generating reports, viewing employee/payroll info) - Enforce role-based access by checking authentication results (role/permissions) before executing admin actions (at a high level)	AuthService / Account / Admin (for access control and identity) DataLoaderService (for loading system data) PayrollService (for payroll computations) AttendanceService (for hours/attendance processing) ReportService / CSVWriter (for generating/exporting reports)	The controller encapsulates the sequence of steps for admin actions (prompt → validate → call service → display output). This prevents business rules from being scattered across the UI and keeps admin workflows consistent.

EmployeeController	- Handle employee-facing workflows (employee menu actions) - Coordinate viewing of personal profile, attendance, and payroll results - Request computations from services when needed (instead of calculating inside the controller)	AuthService / Account / Employee (for login context and identity) AttendanceService (for attendance summaries / hour computations) PayrollService (for gross/net computations) ReportService / View output (for showing payslip/report data)	EmployeeController keeps the "user flow" logic in one place while delegating calculations to services. This is a practical use of encapsulation: the controller exposes simple actions ("view payslip") without exposing the underlying payroll computation steps.
Main (Console entry controller)	- Act as the main entry point of the application - Route the user to the correct controller flow (admin vs employee) - Initialize required services/data for the session	AuthService AdminController EmployeeController DataLoaderService	Main encapsulates application startup and routing, so other classes don't need to know "how the program begins" or how roles are directed to different menus.

Encapsulation in the Controller layer is applied by centralizing user workflow logic inside controller classes (e.g., AdminController, EmployeeController). Controllers coordinate input, validation, and service calls while hiding the internal sequence of steps from the presentation layer. Business rules and computations remain encapsulated in service classes, preventing duplication and improving maintainability.

Utility & Support Layer			
Class Name	**Responsibilities**	**Collaborators**	**Design Intent**
CSVReader	- Read CSV files and convert rows into model objects (e.g., Employee, Attendance, Account) - Hide CSV parsing details (splitting columns, handling headers, data conversion)	DataLoaderService Employee Attendance Account (and any other models created from CSV)	CSVReader encapsulates file-reading and parsing rules so the rest of the system doesn't need to know how CSV files are structured or parsed. This reduces duplication and keeps data-loading consistent.
CSVWriter	- Write/export data into CSV format (e.g., payroll reports, summaries) - Hide CSV formatting rules (column order, separators, headers)	ReportService Payroll (data being exported) Employee (data being exported)	CSVWriter encapsulates output formatting so controllers/services can request an export without manually building CSV strings.
DeductionsUtil	- Provide reusable deduction/tax calculations (SSS, PhilHealth, Pag-IBIG, withholding tax) - Expose deduction formulas through stateless helper methods for consistent reuse	PayrollService	Centralizing formulas here is a practical form of encapsulation: it hides the deduction "rules" behind methods, preventing the same math from being copied across the system.

Encapsulation in the Utility & Support layer is applied by isolating reusable helper functions into dedicated classes such as CSVReader, CSVWriter, and DeductionsUtil. These utilities hide low-level details (file parsing, output formatting, and formula implementation) behind simple method calls, reducing duplication and keeping the system easier to maintain.

MS2 Considerations: Future Plans to Strengthen Encapsulation	
Future consideration:	**Encapsulation benefit:**
Introduce a dedicated LeaveRequest model to manage employee leave data and workflows.	- Prevents direct modification of leave data by enforcing controlled access and state changes. - Protects payroll and attendance integrity as leave-related rules are applied consistently.
Convert remaining protected fields (e.g., in Person and Employee) to private and expose access exclusively through getters/setters.	- Prevents unintended modification by subclasses. - Gives each class full control over how its internal data is accessed or changed.

Provide wrapper methods in the owning class (e.g., Employee.getHourlyRate()) instead of exposing internal objects where appropriate.	- Limits how external classes interact with internal state. - Makes future changes to internal structure less disruptive.
Ensure that controllers and models remain free of calculation logic as new features are added (e.g., bonuses, overtime rules).	- Keeps business rules in one place. - Reduces duplication and improves maintainability.
Make certain models effectively read-only after creation (e.g., payroll results), exposing only getters once values are computed.	- Prevents accidental modification of finalized data. - Clarifies object lifecycle and responsibility.
Enforce utility-only usage by keeping utility classes stateless and method-focused.	- Maintains a clear separation between domain logic and support logic. - Keeps utilities reusable and predictable.
Expand CRC notes and class-level comments to explicitly state what should not be handled by a class (e.g., “This class does not perform payroll calculations”).	- Reduces misuse of classes as the system evolves.

These future considerations focus on strengthening encapsulation by tightening access control, clarifying class responsibilities, and maintaining clean separation between data, logic, and coordination layers. The proposed improvements are incremental and realistic, ensuring that the system remains maintainable and extensible as new features are introduced in subsequent milestones.

ABSTRACTION

Service & Logic Layer			
Class Name	**Responsibilities**	**Collaborators**	**Design Intent**
AuthService	- Authenticate a user based on provided credentials - Determine the user’s role for access control - Provide authentication results to controllers	Account UserRole Admin Employee	AuthService abstracts the authentication process by exposing a simple login/validation operation while hiding credential verification and role determination details from controllers.
PayrollService	- Generate payroll results for an employee - Compute gross pay, deductions, and net pay - Provide payroll outcomes in a consistent, usable form	Employee Compensation AttendanceService Payroll DeductionsUtil	PayrollService abstracts payroll computation by exposing high-level payroll operations while hiding calculation steps, deduction formulas, and processing order.
AttendanceService	- Calculate total hours worked for an employee - Convert attendance records into payroll-usable values	Attendance Employee	AttendanceService abstracts time computation by hiding raw attendance processing logic and exposing only summarized work-hour information.
EmployeeService	- Retrieve and manage employee-related information - Provide employee data to othe, structured system components/service methods	Employee ContactInfo GovernmentID Position Compensation	EmployeeService abstracts employee data access by providing structured operations instead of exposing raw data collections to controllers.

DataLoaderService	- Load required system data for payroll processing - Prepare validated model objects for use by the system	CSVReader Employee Attendance Account	DataLoaderService abstracts data loading by allowing other classes to work with model objects without knowing data source formats or parsing steps.
ReportService	- Generate payroll-related reports - Coordinate preparation of payroll data for output	Payroll Employee PayrollService CSVWriter	ReportService abstracts report generation by exposing report creation operations while hiding formatting and output preparation details.

Model Elements			
Class Name	**Responsibilities**	**Collaborators**	**Design Intent**
UserRole (enum)	- Represent allowed user roles in the system (e.g., ADMIN, EMPLOYEE)	Account AuthService Controllers	UserRole abstracts authorization into a small set of role categories, allowing access control decisions without exposing identity or credential details.

Abstraction is applied by exposing high-level operations through service classes such as AuthService, PayrollService, AttendanceService, EmployeeService, and ReportService. These services hide internal implementation details including authentication rules, payroll calculations, attendance processing, and data loading while providing simple, meaningful operations for controllers to use. Supporting abstractions such as the UserRole enum further simplify system behavior by representing access control using role categories rather than implementation-specific checks.

MS2 Considerations: Future Plans to Strengthen Abstraction

Future consideration:	Abstraction benefit:
Create an AuthResult object to replace boolean login returns by packaging success/failure, role, and user/session data into one result. So that instead of returning/handling multiple values during login, return one object: success (true/false) role (UserRole) user (Employee/Admin or userId)	Controllers handle one clear result (success + role + user) without needing to know internal authentication steps or manage multiple return values.
Create a Payslip/PayrollSummary object to replace scattered payroll output values with one domain result model. After payroll is computed, store the output in one simple model: employee ID/name pay period gross pay total deductions net pay	Controllers request a payslip object instead of manually collecting values, and payroll output can later be shown in different formats (console/CSV) without changing payroll logic.

Add a simple validator class for inputs Add InputValidator or PayrollValidator to handle checks like: missing employee ID invalid dates / months missing attendance records	Services stay focused on doing the main job (compute payroll, generate reports), while validation is handled in one place, keeping logic cleaner and more consistent.

These MS2 considerations strengthen abstraction by increasing the use of high-level operations and reducing exposure to implementation details such as raw data access, deduction formulas, and multi-step workflows. By introducing small domain-level result objects (e.g., authentication results and payslip summaries), improving service method intent, and optionally abstracting deduction and data access logic, the system becomes easier to maintain and extend while keeping controller code simple.

INHERITANCE

Class Name	Responsibilities	Collaborators	Design Intent
Person (Superclass)	- Store basic identity information (first name, last name) - Provide access to identity details shared by all people-related system users	Employee (inherits from Person)	Person serves as the common base class so identity fields and behavior are defined once and reused by employee-related classes.
Employee (Subclass of Person)	- Inherit identity information from Person - Store employee-specific profile information (e.g., employee number, employment details) - Link to employee-related models (ContactInfo, GovernmentID, Position, Compensation)	Person (inherits from) ContactInfo GovernmentID Position Compensation Attendance Payroll	Employee extends Person to reuse shared identity attributes and adds employee-specific data and relationships, reducing duplication and improving organization.
Admin (Subclass of Employee)	- Inherit all employee identity/profile data and behavior from Employee - Represent an administrative user with additional admin-specific identification (e.g., username) - Enable separation between standard employee access and admin-level access	Employee (inherits from) Account AuthService (uses role/access logic)	Admin extends Employee to reuse employee data while specializing the object as an administrator, supporting a clear “Admin is-an Employee” relationship.

Inheritance is applied through a clear class hierarchy where Employee extends Person, and Admin extends Employee. This forms an “is-a” relationship that allows shared identity fields and behavior to be defined once in Person, reused in Employee, and further specialized in Admin. This improves code reuse and keeps related concepts organized in a logical hierarchy.

MS2 Considerations: Future Plans to Strengthen Inheritance

Future consideration:	Inheritance benefit:

Ensure inherited fields remain encapsulated via getters/setters Keep base-class fields private and access them through methods.	Prevents subclasses from directly modifying base state in unintended ways, improving maintainability.
Avoid deep inheritance by limiting role specialization to essential “is-a” relationships, and prefer composition for new capabilities (e.g., permissions, reporting access).	Keeps the hierarchy shallow and flexible, reducing rigidity and preventing changes in one parent class from unintentionally affecting many subclasses.
Avoid “forcing” inheritance when composition is better Use inheritance only for true “is-a” relationships; otherwise use composition (has-a).	Keeps the hierarchy meaningful and prevents bloated parent classes
The current system applies inheritance in a simple and effective manner through the Person → Employee → Admin hierarchy. This structure already supports code reuse and logical organization. For Milestone 2, the following considerations aim to strengthen and future-proof the inheritance structure as the system grows, while keeping the design maintainable and aligned with object-oriented principles.	

POLYMORPHISM

Class Name	Responsibilities	Collaborators	Design Intent
Person (Polymorphic Superclass)	- Represent a generic “person” identity used across the system - Provide shared identity behavior that subclasses reuse	Employee (subclass)	Person provides a common type so code can work with “people” generally (e.g., store/display identity) while actual objects may be Employee instances.
Employee (Polymorphic Superclass + Subclass)	- Represent a general employee user in the system - Serve as the common type for employee-related operations and access flows	Person (superclass) Admin (subclass) Services that work on employees (e.g., AuthService, PayrollService, AttendanceService)	Employee enables code to treat different employee-like users uniformly. For example, an Admin can be passed anywhere an Employee is expected, because Admin is an Employee.
Admin (Polymorphic Subclass)	- Represent an administrative user as a specialized employee - Reuse employee identity/profile data via inheritance - Override shared methods where admin output/behavior differs (e.g., toString() / getInfoSummary())	Employee (superclass) Account / AuthService (for role-based access decisions)	Admin objects can be handled through the Employee type in shared logic, while still being recognized as admin users through role checks (e.g., UserRole) and controller routing.
AuthService (Polymorphism Usage Point)	- Authenticate users and determine the correct access path (admin vs employee) - Work with users at a higher level (role/type) without requiring controllers to know login internals	Account UserRole Employee / Admin	AuthService operates on the parent type (Employee) and returns role/context results, allowing the same authentication flow to support both Employee and Admin objects via dynamic dispatch or role-based routing.
UserRole (Support for Polymorphic Behavior)	- Represent the allowed system roles (e.g., ADMIN, EMPLOYEE)	Account AuthService Controllers	While UserRole isn’t polymorphism by itself, it supports polymorphic flows by letting controllers handle different user types through the same high-level routing logic.
Polymorphism is applied through subtype polymorphism in the class hierarchy Person → Employee → Admin. This allows the system to treat specialized objects (e.g., Admin) as their parent type (Employee) in shared logic, while still supporting specialized behavior when needed (via role checks and, where applicable, overridden methods). Java supports runtime polymorphism through method overriding and dynamic dispatch.			

MS2 Considerations: Future Plans to Strengthen Polymorphism	
Future consideration:	**Polymorphism benefit:**
Introduce interface-based polymorphism by creating a Payable interface so PayrollService can compute payroll for any payable entity (not only Employee).	- Allows PayrollService to operate on a common interface w/ new payable types without changing core logic. - Reduces tight coupling between payroll computation and the Employee class, improving flexibility and extensibility.
Define a common Authenticatable (or UserAccount) interface for login-capable users to unify authentication flows beyond Employee/Admin.	- Enables AuthService to authenticate different user types through one interface with minimal controller changes. - Supports adding future roles without expanding the inheritance hierarchy

MAPÚA MALAYAN DIGITAL COLLEGE

A college under Mapúa Malayan Colleges Laguna

MO-IT110 Object-Oriented Programming

Method Dictionary

Select a layer to view its Method Dictionary (Click the cell below):

Model Layer	Controller Layer	Service Layer	Utility Layer	Auth Layer			

Model Layer

Class	Method Name	Purpose	OOP Principle	Visibility	Parameters	Return Type	Validation Rule / Notes
Account.java	Account(String username, String hashedPassword, String role)	Constructor to initialize a new user account with credentials and access level.	Encapsulation	Public	String username String hashedPassword String role	N/A	Should ensure parameters are not null or empty strings.
	getUsername()	Retrieves the account's unique identifier.	Encapsulation	Public	None	String	None
	getHashedPassword()	Retrieves the secured (hashed) password string.	Encapsulation	Public	None	String	Returns a one-way cryptographic hash; plain-text passwords are never stored or returned (Ensures sensitive data is never stored in plain-text).
	getRole()	Retrieves the authorization level (Admin vs. Employee).	Encapsulation	Public	None	role	Used for role-based access control (RBAC).
Admin.java	Admin(int employeeNumber, String firstName, String lastName, LocalDate birthday, ContactInfo contact, GovernmentID governmentID, Position position, Compensation compensation, String status, String password, String username)	Initializes Admin via parent class.	Inheritance	Public	int employeeNumber String firstName String lastName LocalDate birthday ContactInfo contact GovernmentID governmentID Position position Compensation compensation String status String password String username	N/A	Uses super() to initialize inherited fields; ensures all 11 data points are captured upon creation.
	getUsername()	Retrieves the unique administrative username.	Encapsulation	Public	None	String	Provides controlled access to the private username field.
	getAdminInfo()	Generates a formatted string representing the Admin's identity.	Abstraction	Public	None	String	Simplifies data access by combining getFullName() and username into a single output.
	authenticate(String inputUsername, String inputPassword)	Validates credentials by checking the username locally and the password via the parent class.	Polymorphism	Public	-String inputUsername -String inputPassword	boolean	Overrides/Extends base logic; uses super. authenticate(inputPassword) for password verification.

Attendance.java	Attendance(Employee employee, LocalDate date, LocalTime timeIn, LocalTime timeOut)	Initializes a record linking a specific employee to their work hours on a specific date.	Encapsulation	Public	-Employee employee -LocalDate date -LocalTime timeIn -LocalTime timeOut	N/A	Ensures that an attendance record cannot exist without a valid Employee object and timestamp.
	getEmployee()	Retrieves the Employee object associated with this attendance record.	Encapsulation	Public	None	Employee	Returns the full Employee object, allowing access to employee details.
	getDate()	Retrieves the date of the attendance record.	Encapsulation	Public	None	LocalDate	Used for filtering and payroll period calculations.
	getTimeIn()	Retrieves the recorded start time.	Encapsulation	Public	None	LocalTime	Essential for calculating total hours worked.
	getTimeOut()	Retrieves the recorded end time.	Encapsulation	Public	None	LocalTime	Validation: timeOut should logically occur after timeIn.
	toString()	Overrides the default Object string representation to provide a formatted summary.	Polymorphism	Public	None	String	Uses employee.getFullName() to abstract the complexity of name concatenation.
Compensation.java	Compensation(double basicSalary, double riceSubsidy, double phoneAllowance, double clothingAllowance, double semiGross, double hourlyRate)	Initializes the financial profile for an employee with all standard allowances and rates.	Encapsulation	Public	double basicSalary double riceSubsidy double phoneAllowance double clothingAllowance double semiGross double hourlyRate	N/A	None
	getBasicSalary()	Retrieves the fixed monthly base pay.	Encapsulation	Public	None	double	Should be ≥ 0
	setBasicSalary(double basicSalary)	Updates the monthly base pay.	Encapsulation	Public	double basicSalary	void	Value must be ≥ 0
	getRiceSubsidy()	Retrieves the monthly rice allowance.	Encapsulation	Public	None	double	Should be ≥ 0
	setRiceSubsidy(double riceSubsidy)	Updates the rice allowance amount.	Encapsulation	Public	double riceSubsidy	void	Value must be ≥ 0
	getPhoneAllowance()	Retrieves the communication allowance.	Encapsulation	Public	None	double	Should be ≥ 0
	setPhoneAllowance(double phoneAllowance)	Updates the communication allowance.	Encapsulation	Public	double phoneAllowance	void	Value must be ≥ 0
	getClothingAllowance()	Retrieves the clothing subsidy.	Encapsulation	Public	None	double	Should be ≥ 0
	setClothingAllowance(double clothingAllowance)	Updates the clothing subsidy.	Encapsulation	Public	double clothingAllowance	void	Value must be ≥ 0
	getSemiGross()	Retrieves the pre-deduction salary sum.	Encapsulation	Public	None	double	Computed or assigned value
	setSemiGross(double semiGross)	Manually sets or updates the semi-gross total.	Encapsulation	Public	double semiGross	void	Should be ≥ 0
	getHourlyRate()	Retrieves the rate used for overtime and late deductions.	Encapsulation	Public	None	double	Should be ≥ 0
	setHourlyRate(double hourlyRate)	Updates the calculated hourly rate.	Encapsulation	Public	double hourlyRate	void	Value must be > 0
ContactInfo.java	ContactInfo(String address, String phoneNumber)	Initializes the contact profile with physical address and mobile number.	Encapsulation	Public	-String address -String phoneNumber	N/A	Parameters should not be null or empty to ensure reachable contact data.
	getAddress()	Retrieves the stored physical address.	Encapsulation	Public	None	String	Provides read access to the private address field.
	setAddress(String address)	Updates the physical address of the employee.	Encapsulation	Public	String address	void	New value must not be empty; should follow standard address formatting.
	getPhoneNumber()	Retrieves the stored contact number.	Encapsulation	Public	None	String	Provides read access to the private phoneNumber field.
	setPhoneNumber(String phoneNumber)	Updates the contact phone number.	Encapsulation	Public	String phoneNumber	void	New value should follow local or international telephony patterns (e.g., 11 digits for PH mobile).

Employee.java	Employee(int employeeNumber, String firstName, String lastName, LocalDate birthday, ContactInfo contact, GovernmentID governmentID, Position position, Compensation compensation, String status, String password)	Full initialization of an employee record.	Inheritance	Public	int employeeNumber String firstName String lastName LocalDate birthday ContactInfo contact GovernmentID governmentID Position position Compensation compensation String status String password	N/A	Uses super() to set name fields in the Person class.
	Employee(String firstName, String lastName)	Constructor Overloading: Minimal setup for supervisor references without full data.	Polymorphism	Public	-String firstName -String lastName	N/A	Initializes employeeNumber to -1 and other objects to null.
	getEmployeeNumber()	Retrieves the unique corporate ID	Encapsulation	Public	None	int	Provides read access to protected ID field.
	setEmployeeNumber(int newEmpNum)	Updates the corporate ID	Encapsulation	Public	int newEmpNum	void	New ID must be ≥ 0
	getBirthday()	Retrieves the date of birth.	Encapsulation	Public	None	LocalDate	Used for age and benefit eligibility.
	setBirthday(LocalDate birthday)	Updates the birth date.	Encapsulation	Public	LocalDate birthday	void	Must be a valid date in the past.
	getContact()	Accesses the full contact info object.	Encapsulation	Public	None	ContactInfo	Returns an object containing address/phone.
	getGovernmentID()	Accesses ID numbers (SSS, TIN, etc.).	Encapsulation	Public	None	GovernmentID	Accesses the associated government ID object.
	getPosition()	Accesses job title and supervisor info.	Encapsulation	Public	None	Position	Accesses the associated position object.
	getCompensation()	Accesses the financial details object.	Encapsulation	Public	None	Compensation	Accesses the associated salary object.
	getStatus()	Checks employment status (e.g., Regular).	Encapsulation	Public	None	String	Returns current work status string.
	setStatus(String status)	Updates the employment status.	Encapsulation	Public	String status	void	Should match system constants.
	getHourlyRate()	Retrieves rate by delegating to Compensation.	Abstraction	Public	None	double	Hides the fact that this data lives in Compensation.
	setHourlyRate(double hourlyRate)	Updates rate via the Compensation object.	Abstraction	Public	double hourlyRate	void	value must be ≥ 0
	getBasicInfo()	Returns a high-level summary string.	Abstraction	Public	None	String	Simplifies complex data for display lists.
	toString()	Returns a massive formatted profile string.	Encapsulation	Public	None	String	Overrides standard Object method; includes null-checks.
GovernmentID.java	GovernmentID(String sss, String philhealth, String tin, String pagibig)	Initializes the collection of statutory government identifiers for an employee.	Encapsulation	Public	String sss String philhealth String tin String pagibig	N/A	Parameters should not be null; should follow standard agency formats.
	getSss()	Retrieves the Social Security System number.	Encapsulation	Public	None	String	Provides read access to the private sss field.
	setSss(String sss)	Updates the stored SSS number.	Encapsulation	Public	String sss	void	Should validate for a 10-digit numeric format.
	getPhilhealth()	Retrieves the PhilHealth ID number.	Encapsulation	Public	None	String	Accesses the private philhealth field.
	setPhilhealth(String philhealth)	Updates the PhilHealth ID number.	Encapsulation	Public	String philhealth	void	Should validate for a 12-digit numeric format.
	getTin()	Retrieves the Tax Identification Number.	Encapsulation	Public	None	String	Accesses the private tin field.
	setTin(String tin)	Updates the stored TIN.	Encapsulation	Public	String tin	void	Should validate for a 9 to 12-digit format.
	getPagibig()	Retrieves the Pag-IBIG number.	Encapsulation	Public	None	String	Accesses the private pagibig field.
	setPagibig(String pagibig)	Updates the Pag-IBIG number.	Encapsulation	Public	String pagibig	void	Should validate for a 12-digit numeric format.

Payroll.java	getEmployee()	Retrieves the Employee object associated with this payroll record.	Encapsulation	Public	None	Employee	Returns the reference to the specific employee being processed.
	setEmployee(Employee employee)	Links an employee to a new payroll calculation instance.	Encapsulation	Public	Employee employee	void	The Employee object must not be null.
	getSss()	Retrieves the calculated SSS deduction.	Encapsulation	Public	None	double	Accesses the private sss contribution amount.
	setSss(double sss)	Manually sets or updates the SSS deduction amount.	Encapsulation	Public	double sss	void	Value must be ≥ 0.
	getPhilhealth()	Retrieves the calculated PhilHealth deduction.	Encapsulation	Public	None	double	Accesses the private philhealth contribution amount.
	setPhilhealth(double philhealth)	Updates the PhilHealth deduction amount.	Encapsulation	Public	double philhealth	void	Value must be ≥ 0.
	getPagibig()	Retrieves the calculated Pag-IBIG deduction.	Encapsulation	Public	None	double	Accesses the private pagibig contribution amount.
	setPagibig(double pagibig)	Updates the Pag-IBIG deduction amount.	Encapsulation	Public	double pagibig	void	Value must be ≥ 0.
	getTax()	Retrieves the calculated Withholding Tax.	Encapsulation	Public	None	double	Accesses the private tax field.
	setTax(double tax)	Updates the calculated tax amount.	Encapsulation	Public	double tax	void	Value must be ≥ 0.
	getNetSalary()	Retrieves the final "take-home" pay.	Encapsulation	Public	None	double	Represents Gross Salary - Total Deductions.
	setNetSalary(double netSalary)	Updates the net salary amount.	Encapsulation	Public	double netSalary	void	Value must be ≥ 0.
	getGrossSalary()	Retrieves the total earnings before deductions.	Encapsulation	Public	None	double	Includes base pay and all taxable allowances.
	setGrossSalary(double grossSalary)	Updates the gross salary amount.	Encapsulation	Public	double grossSalary	void	Value must be ≥ 0.
	computeDeductions()	Logic placeholder for calculating all statutory contributions.	Abstraction	Public	None	double	Returns the sum of SSS, PhilHealth, Pag-IBIG, and Tax.
	generatePayslip()	Logic placeholder for formatting and outputting the final payroll report.	Abstraction	Public	None	void	Consolidates all model data into a user-readable format.
Person.java	Person(String firstName, String lastName)	Initializes the fundamental identity of a person.	Inheritance	Public	String firstName, String lastName	N/A	Base constructor called by all subclasses via super().
	getFirstName()	Retrieves the person's given name.	Encapsulation	Public	None	String	Provides read access to the protected firstName field.
	setFirstName(String firstName)	Updates the person's given name.	Encapsulation	Public	String firstName	void	New value must not be null or empty strings.
	getLastName()	Retrieves the person's family name.	Encapsulation	Public	None	String	Provides read access to the protected lastName field.
	setLastName(String lastName)	Updates the person's family name.	Encapsulation	Public	String lastName	void	New value must not be null or empty strings.
	getFullName()	Combines the first and last name into a single string.	Abstraction	Public	None	String	Simplifies data retrieval; hides the concatenation logic from the user.
Position.java	Position(String position, Employee supervisor)	Initializes a job role with its title and designated supervisor.	Encapsulation	Public	String position, Employee supervisor	N/A	supervisor can be null if the position is the highest in the hierarchy (e.g., CEO).
	getPosition()	Retrieves the job title.	Encapsulation	Public	None	String	Accesses the private position name.
	setPosition(String position)	Updates the job title.	Encapsulation	Public	String position	void	New title must not be null or an empty string.
	getSupervisor()	Retrieves the Employee object acting as the supervisor for this role.	Encapsulation	Public	None	Employee	Used to traverse the corporate hierarchy.
	setSupervisor(Employee supervisor)	Assigns a new supervisor to the position.	Encapsulation	Public	Employee supervisor	void	Used during departmental restructuring or promotions.
	toString()	Returns the job title as the string representation of the object.	Polymorphism	Public	None	String	Overrides standard Object method for cleaner UI display.

Controller Layer

Class	Method Name	Purpose	OOP Principle	Visibility	Parameters	Return Type	Validation Rule / Notes
AdminController.java	viewEmployeeInfo(int empNum)	Requests the system to find and display a specific employee's profile.	Abstraction	Public	int empNum	void	empNum must be verified to exist in the database.
	generatePayrollForAll()	Triggers the bulk calculation of payroll for the entire staff.	Abstraction	Public	None	void	Should check if attendance data for the period is complete before running.
EmployeeController.java	EmployeeController()	Default constructor for the employee-level logic handler.	Encapsulation	Public	None	N/A	Default Constructor: Creates the controller instance for standard user actions.

Service Layer							
Class	**Method Name**	**Purpose**	**OOP Principle**	**Visibility**	**Parameters**	**Return Type**	**Validation Rule / Notes**
AttendanceService.java	AttendanceService (List<Attendance> attendanceRecords)	Initializes the service by calculating the minimum and maximum date ranges from the provided records.	Encapsulation	Public	List<Attendance> attendanceRecords	N/A	Automatically sets range boundaries; defaults to LocalDate.MIN/MAX if list is empty.
	isWithinAttendanceRange (LocalDate date)	Helper method to check if a specific date exists within the system's known timeline.	Encapsulation	Private	LocalDate date	boolean	Returns false if date is before minAttendanceDate or after maxAttendanceDate.
	isWithinAttendanceRange (YearMonth month)	**Polymorphism (Overloading): Checks if an entire month falls within the recorded data range.**	Polymorphism	Private	YearMonth month	boolean	Converts month to start/end dates to validate against class boundaries.
	calculateMonthlyHours (Employee employee, YearMonth month)	Sums total work hours for a specific employee within a given month.	Abstraction	Public	Employee employee, YearMonth month	double	Throws IllegalArgumentException if month is out of range; results must be ≥ 0.
	calculateFixedWeekHours (Employee employee, LocalDate referenceDate)	Calculates total hours for a specific week cycle based on a reference date.	Abstraction	Public	Employee employee, LocalDate referenceDate	double	Uses calculateFixedWeekRange to identify the specific 5-day work cycle.
	calculateFixedWeekRange (LocalDate referenceDate)	Business logic to determine the 5-day work window based on a 2024 anchor date.	Encapsulation	Public	LocalDate referenceDate	LocalDate[]	Uses a 7-day cycle logic (5 work days + 2 gap days) to find the start and end dates.
AuthService.java	loadAccounts()	Populates the internal account map by reading and parsing the accounts.csv file.	Abstraction	Public	None	void	Throws IOException if file reading fails; skips header and malformed lines.
	authenticate(String username, String password)	Validates login attempts by comparing the hash of the input password with the stored hash.	Abstraction	Public	String username, String password	boolean	Returns false if username does not exist or password hash does not match.
	getRole(String username)	Retrieves the access level assigned to a specific username.	Encapsulation	Public	String username	String	Returns null if the user is not found in the accountMap.
	hashPassword(String password)	Converts plain-text passwords into a secure SHA-256 hexadecimal string.	Encapsulation	Public (Static)	String password	String	Uses MessageDigest; critical for preventing the storage of plain-text credentials.
DataLoaderService.java	loadAllData()	Orchestrates the sequential loading of employee and attendance data from external CSV files.	Abstraction	Public	None	void	Critical Path: Employees must be loaded first so they can be linked to attendance records.
	getEmployees()	Provides access to the list of loaded employee objects.	Encapsulation	Public	None	List<Employee>	Returns the private employees collection.
	getAttendanceRecords()	Provides access to the list of loaded attendance records.	Encapsulation	Public	None	List<Attendance>	Returns the private attendanceRecords collection.
	EmployeeService (List<Employee> employeeList)	Initializes the service with a list of employee objects managed in memory.	Encapsulation	Public	List<Employee> employeeList	N/A	Injects the data dependency into the service instance.

EmployeeService.java	printAllEmployeeData()	Iterates through the list to print a detailed summary of every employee.	Polymorphism	Public	None	void	Leverages Employee.toString() (Polymorphism) for standardized output.
	printEmployeeData(Employee employee)	Displays a formatted, category-based view of a specific employee's details.	Abstraction	Public	Employee employee	void	Includes null-checks for ContactInfo and Compensation to prevent runtime errors.
	getAllEmployees()	Provides access to the full internal employee list.	Encapsulation	Public	None	List<Employee>	Returns the private employeeList reference.
	findEmployeeByNumber(int employeeNumber)	Searches the list for a match based on the unique corporate ID.	Abstraction	Public	int employeeNumber	Employee	Uses Stream API; returns null if the ID does not exist in the collection.
	findEmployeeByFullName(String fullName)	Searches for an employee using their full name, ignoring capitalization differences.	Abstraction	Public	String fullName	Employee	Uses equalsIgnoreCase to ensure flexible user search capabilities.
PayrollService.java	PayrollService (AttendanceService attendanceService)	Injects the required attendance logic dependency into the payroll engine.	Encapsulation	Public	AttendanceService attendanceService	N/A	Ensures the service has access to hours-worked data upon initialization.
	calculateMonthlySalary (Employee employee, YearMonth month)	Calculates total gross pay including hourly wages and full monthly allowances.	Abstraction	Public	Employee employee, YearMonth month	double	Aggregates base salary and fixed benefits; value must be ≥ 0.
	calculateWeeklySalary (Employee employee, LocalDate referenceDate)	Calculates gross pay for a 7-day period, including prorated (1/4) allowances.	Abstraction	Public	Employee employee, LocalDate referenceDate	double	Uses weekly hours from AttendanceService; value must be ≥ 0.
	calculateNetSalary(Employee employee, YearMonth month)	Orchestrates the full monthly payroll process: Gross - (SSS + PhilHealth + Pag-IBIG + Tax).	Abstraction	Public	Employee employee, YearMonth month	double	Uses DeductionsUtil for statutory math; handles complex taxable income logic.
	calculateNetWeeklySalary (Employee employee, LocalDate referenceDate)	Executes a complete weekly payroll run with prorated statutory deductions.	Abstraction	Public	Employee employee, LocalDate referenceDate	double	Ensures financial consistency across different pay frequencies.
ReportService.java	ReportService(PayrollService payrollService, AttendanceService attendanceService)	Initializes the service with the required payroll and attendance logic engines.	Encapsulation	Public	PayrollService, AttendanceService	N/A	Injects necessary dependencies for report generation.
	printPayrollReport (List<Employee> employees, YearMonth month)	Generates a high-level console summary of gross and net pay for a list of employees.	Abstraction	Public	List<Employee> employees, YearMonth month	void	Iterates through the collection to provide a bulk overview.
	printPayrollForEmployee (Employee emp, Compensation compensation, YearMonth month)	Provides a detailed, itemized monthly breakdown of earnings and deductions for a single employee.	Abstraction	Public	Employee emp, Compensation compensation, YearMonth month	void	Explicitly displays statutory deductions (SSS, PhilHealth, etc.) and withholding tax.
	printWeeklyPayrollForEmployee (Employee emp, Compensation compensation, LocalDate referenceDate)	Provides a detailed itemized breakdown for a specific 7-day cycle.	Abstraction	Public	Employee emp, Compensation compensation, LocalDate referenceDate	void	Uses attendanceService to identify the specific start and end dates of the week.
	exportPayrollReportToCSV (List<Employee> employees, YearMonth month, String filePath)	Persists the payroll data to an external CSV file for auditing or external use.	Abstraction	Public	List<Employee> employees, YearMonth month, String filePath	void	Advanced File I/O: Automatically creates parent directories and handles absolute vs. relative paths.

Utility (Util) Layer							
Class	**Method Name**	**Purpose**	**OOP Principle**	**Visibility**	**Parameters**	**Return Type**	**Validation Rule / Notes**

CSVReader.java	readEmployeesFromCSV(String filepath)	Performs a two-pass load: first creates employee objects, then resolves and links supervisor relationships.	Abstraction	Public (Static)	String filepath	List<Employee>	Throws RuntimeException on failure; skips header; handles "none" or blank supervisors.
	findEmployeeByFullName (List<Employee> employees, String fullName)	Private helper to locate an existing employee object using a "FirstName LastName" string.	Encapsulation	Private (Static)	List<Employee>, String fullName	Employee	Case-insensitive; returns null if no match is found.
	findEmployeesSupervisor (List<Employee> employees, String supervisorField)	Specialized search for supervisors using the "LastName, FirstName" format found in the CSV.	Encapsulation	Private (Static)	List<Employee>, String supervisorField	Employee	Normalizes spaces and splits by commas to ensure accurate matching.
	parseDoubleSafe(String s)	Converts raw CSV text into a double while removing formatting artifacts like quotes and commas.	Abstraction	Private (Static)	String s	double	Defensive: Returns 0.0 if parsing fails to prevent system crashes.
	readAttendanceFromCSV(String filepath, List<Employee> employees)	Parses time logs and uses the provided employee list to associate logs with actual objects.	Abstraction	Public (Static)	String filepath, List<Employee> employees	List<Attendance>	Skips records where the employeeNumber doesn't match a loaded employee.
CSVWriter.java	appendEmployeeToCSV (Employee employee)	Adds a single new employee record to the end of the existing CSV file.	Abstraction	Private (Static)	Employee employee	void	Automatically adds the header row if the file is new or empty.
	formatDouble(double value)	Formats numeric values as plain strings to maintain CSV consistency.	Encapsulation	Private (Static)	double value	String	Ensures no commas or quotes are added to numeric data.
	updateEmployeeInCSV (Employee updatedEmployee)	Finds an existing record by ID, replaces it in memory, and rewrites the file.	Abstraction	Public (Static)	Employee updatedEmployee	void	Throws IOException if the employee ID does not exist.
	deleteEmployeeFromCSV(int employeeNumber)	Removes an employee from the data set and updates the persistent storage.	Abstraction	Public (Static)	int employeeNumber	void	Uses removeIf for efficient filtering; throws exception if ID is missing.
	writeAllEmployeesToCSV (List<Employee> employees)	Overwrites the entire CSV file with the current list of employee objects.	Abstraction	Public (Static)	List<Employee> employees	void	Used as the final step for both Update and Delete operations.
DeductionsUtil.java	calculatePhilhealth(double grossSalary)	Computes the employee's 50% share of the PhilHealth premium based on the current 3% rate.	Abstraction	Public (Static)	double grossSalary	double	Includes caps at 300 (min) and 1800 (max). Throws IllegalArgumentException if salary is negative.
	calculatePagIbig(double finalSalary)	Determines the Pag-IBIG contribution using a tiered rate (1% or 2%) based on income levels.	Abstraction	Public (Static)	double finalSalary	double	Capped at 100.00; returns 0 for salaries below 1000.
	calculateSSS(double finalSalary)	Performs a range-based lookup against a 45-tier bracket table to find the SSS contribution.	Abstraction	Public (Static)	double finalSalary	double	Uses a 2D array for efficiency; returns the employee's 50% share.
	calculateWithholdingTax(double taxableIncome)	Applies the BIR progressive tax schedule to determine the monthly withholding tax.	Abstraction	Public (Static)	double taxableIncome	double	Implements 6 distinct tax brackets ranging from 0% to 35%.

Auth Layer							
Class	**Method Name**	**Purpose**	**OOP Principle**	**Visibility**	**Parameters**	**Return Type**	**Validation Rule / Notes**
UserRole.java	EMPLOYEE	Represents a standard user with restricted access to personal data only.	Encapsulation	Public (Static)	N/A	UserRole	Constant used for Role-Based Access Control (RBAC).
	ADMIN	Represents a privileged user with access to administrative tools and bulk payroll.	Encapsulation	Public (Static)	N/A	UserRole	Constant used to bypass standard user restrictions.
	valueOf(String name)	Automatically generated method to convert a String into a UserRole enum.	Abstraction	Public (Static)	String	UserRole	Technical Challenge: Throws IllegalArgumentException if the string does
	values()	Automatically generated method that returns an array of all roles in the enum.	Abstraction	Public (Static)	None	UserRole[]	Useful for populating login dropdowns or validation loops.

MAPÚA MALAYAN DIGITAL COLLEGE

A college under Mapúa Malayan Colleges Laguna

MO-IT110 Object-Oriented Programming

Architecture Overview

Week	OOP Principle	Architecture Explanation
2	Encapsulation	In the MotorPH Payroll System, sensitive identifiers including Government IDs and financial data such as hourly rates are secured with private access modifiers to prevent unauthorized modification. Interaction with this data is strictly channeled through public getters and setters, which serve as controlled gateways for validation logic. Furthermore, core statutory calculations are encapsulated within the DeductionsUtil utility class, shielding complex formulas from the rest of the system. While most data is private, certain identity attributes in the Person class use protected visibility to allow for efficient inheritance while still restricting access from unrelated classes.
3	Abstraction	Abstraction is implemented through a specialized Service Layer that seperates high-level business logic from implementation details. The PayrollService, AuthService, and AttendanceService provide simplified methods such as calculateNetSalary() and authenticate(), allowing the system to execute complex workflows without exposing the underlying data-mapping or hashing processes. Additionally, the UserRole enum abstracts system access into logical categories, enabling the application to manage permissions based on roles (ADMIN vs. EMPLOYEE) rather than hard-coded string, ensuring a cleaner and more maintainable interface. Futhermore, the architecture plans to transition from primitive return types to specialized Abstraction Objects like AuthResult to better encapsulate complex system states.
4	Polymorphism	Polymorphism is primarily applied through subtype polymorphism within the Person → Employee → Admin hierarchy. This allows the system to treat specialized Admin objects as general Employee types in shared processes like payroll calculation, reducing code duplication. Furthermore, the system utilizes method overloading in the AttendanceService (e.g., isWithinAttendanceRange), enabling the same method to handle different parameters like specific dates or entire months. These behaviors allow the system to remain flexible and handle diverse user roles and data types through a consistent set of operations. To further decouple the system, the architecture includes a roadmap to introduce interface-based polymorphism via a Payable interface, allowing the payroll logic to extend to new entity types without
5	Inheritance	Inheritance serves as the structural foundation of the MotorPH data model by establishing a clear 'is-a' relationship across classes. The Person superclass defines fundamental identity attributes that are inherited by Employee, which is then further specialized by Admin class to include security credentials. This hierarchy ensures that common attributes like names and birthdates are defined in a single parent class, eliminating redundancy. By utilizing super() constructors, child classes are reliably initialized with their full inherited state, creating a stable and extensible architecture than can easily accommodate new employee roles in the future.

MAPÚA MALAYAN DIGITAL COLLEGE

A college under Mapúa Malayan Colleges Laguna

MO-IT110 Object-Oriented Programming

Team Contributions Table

How to Use This Tab

Each member fills ONE ROW for each task/artifact your team worked on that week. You will describe your own contribution in the Contribution Description column.

After all members write their contribution, every other team member must rate that person's contribution individually.

Fill your own rating in the Self-Rating column. Fill one rating per peer across the Peer Rating columns.

Do NOT give perfect scores automatically. Provide fair and honest assessment.

Ratings should follow the scale below:

***0** – No contribution / did not participate*

***1** – Minimal contribution / incomplete*

***2** – Expected contribution / satisfactory*

***3** – Strong contribution / exceeded expectations*

Member Name	Contribution Description (self-written)	Self-Rating (0–3)	Peer Rating – Member A	Peer Rating – Member B	Peer Rating – Member C
Joemel Bataga					
Maria Alissa Bernales					
Grazielle Felice Canoza					
Shane Rivera					